

Self-Contradiction Detector

A rule-based engine that catches confident AI errors — fluent answers that secretly conflict with themselves

Submitted by: **Jeevan**

Date: June 2026

Internship Role: AI / ML Engineering Intern — Protofine.ai

1. The Problem Statement

1.1 What Protofine Asked For

The Protofine.ai internship brief asked me to build a detector for confident AI errors — specifically the fluent, plausible, wrong kind. Not obvious nonsense, but answers that sound authoritative while secretly being broken.

Exact brief from Protofine.ai

"Build a detector for confident AI errors — the fluent, plausible, wrong kind, not obvious nonsense. Catching the confident ones is the hard part, and it is exactly what matters."

I was given three failure types to choose from:

- **Fabricated statistic** — an AI inventing a number that sounds real
- **Made-up citation** — an AI citing a paper or author that does not exist
- **Self-contradiction** — an AI making two claims in the same answer that logically cannot both be true

1.2 Why I Chose Self-Contradiction

I chose self-contradiction deliberately. Here is my reasoning:

1. **It is the hardest failure type to detect by instinct.** A fabricated stat can sometimes be Googled. A fake citation can be verified against a database. But a self-contradiction that spans two paragraphs — where both halves sound expert-level — is completely invisible unless you reason about the logical relationship between sentences. Readers process text sequentially and almost never catch cross-paragraph conflicts.
2. **It requires reasoning, not retrieval.** A fact-checker is essentially a database lookup. A contradiction detector must understand what claims mean, map their logical relationship, and determine mutual exclusivity. That is a much richer problem — and it is closer to what real AI metacognition looks like.
3. **It is the most dangerous failure mode in practice.** When an AI contradicts itself about a medication dosage, a security policy, or an engineering specification, one of those two claims is wrong — and the user trusts both because the overall answer sounds confident.

1.3 The Specific Difficulty I Was Targeting

The brief said 'catching the confident ones is the hard part.' I designed my test cases specifically around this. Here is the hardest case in my test suite — it looks like a normal expert-level explanation:

Case 3 — The Tricky Case (Confident Wrong)

"Transformer models are inherently stateless — each forward pass processes the input independently with no memory of previous inputs. This is a fundamental architectural property that makes them scalable and parallelizable. The attention mechanism allows transformers to maintain and update a running state across tokens in a sequence, effectively giving the model memory of earlier tokens within the same context window." Both halves sound authoritative. Neither hedges. A human reading quickly would miss it entirely. The engine catches it at score 75/100 HIGH confidence.

2. How I Researched & Understood the Problem

2.1 Perplexity AI — Domain Research

I used Perplexity AI as my primary research tool before writing any code. Specifically to understand:

- What types of AI hallucinations exist and how researchers classify them
- Existing academic work on contradiction detection and textual entailment in NLP
- What makes self-contradictions hard to detect (the inter-sentence reasoning problem)
- Why LLM-based detectors can be unreliable — temperature means the same input gets different outputs across runs
- The critical distinction between nuance and contradiction — which became the most important design constraint in the entire project

Key insight from Perplexity research: the primary failure mode of contradiction detectors is over-flagging — mistaking qualified statements for contradictions. This directly shaped my decision to use specific antonym phrases instead of broad word matching.

2.2 NotebookLM — Deep Understanding

I used Google NotebookLM to upload NLP research papers and generate structured explanations. I specifically used it to understand the NLI (Natural Language Inference) taxonomy — the formal academic framing of how sentences relate to each other: entailment, contradiction, or neutral. This is the theoretical backbone of what my engine detects.

I also used NotebookLM's audio generation feature to create an audio explanation of how sentence-pair contradiction detection works. Listening to a synthesized explanation of a complex topic while thinking through architecture decisions is an effective learning technique I use regularly.

2.3 ChatGPT — Problem Solving & Coding

Used for thinking through the architecture (rule-based vs LLM-based decision), debugging the false-positive problem, and working through edge cases in the shared-keyword overlap logic.

Where ChatGPT gave wrong output

ChatGPT initially suggested using spaCy for NLP processing (dependency parsing, named entity recognition) to improve detection accuracy. I rejected this because it would add a heavy external dependency and make the project harder to run. My decision to stay with pure Python standard library was the right one — zero-dependency tools are significantly easier to adopt.

2.4 Claude.ai — Coding & Architecture

Used for writing the core detector.py rule engine, main.py CLI, test runner, and README. Also used for the reliability analysis — precision vs recall tradeoffs, false-positive risk categories, and what a proper benchmark would require.

Where Claude gave wrong output — and how I fixed it

An early LLM-based prototype using the Claude API as the detector flagged nuanced text as contradictions. 'Machine learning generally improves with more data, but there are diminishing returns' was incorrectly flagged as a contradiction. Both claims are simultaneously true — that is nuance, not conflict. The fix was switching entirely from LLM-based to deterministic rule-based detection, and making antonym pairs deliberately specific. Case 5 in the test suite proves the fix works — it scores 0 every time.

2.5 Gemini — CSS, HTML, JavaScript Styling

Used for the visual design of the HTML demo. The dark sidebar layout, animated SVG score ring, confidence tier color system (green/amber/red), and responsive grid layout were all styled with Gemini's assistance. I specified the design language and information hierarchy; Gemini executed the CSS and HTML structure.

2.6 Jarvis (My Personal AI) — Previous Documentation

I used my own AI assistant Jarvis to access and reference previous documentation and notes I had built up. This helped me retrieve context about Python project structure conventions, CLI design patterns, and README formatting standards without starting from scratch.

2.7 HeyGen.ai — AI Walkthrough Video

Used to generate the AI-presented walkthrough video explaining the project. HeyGen's avatar video generation produced a professional explanation video for the submission.

2.8 Prompting — 100% My Own

All prompts across every tool were written entirely by me. This is the part that cannot be outsourced — knowing what to ask, how to frame the problem, what constraints to specify, what output format to request, when to reject an answer, and when to try a completely different approach. The AI tools executed. I directed.

3. My Thinking & Architecture Decisions

3.1 The Key Decision: Rule-Based Over LLM-Based

My first prototype used an LLM as the detection backend. I identified three structural problems with this approach:

- **Non-determinism:** same input, different outputs across runs (temperature > 0). A reliability tool must be deterministic.
- **API dependency:** requiring an API key adds friction, cost, and a runtime failure point. A tool designed to verify AI output should not itself depend on AI to run.
- **Opacity:** when an LLM flags something, you get a reason in prose. You cannot inspect which rule fired or why. A rule-based system shows exactly what matched.

Decision: switch to a deterministic rule-based engine in pure Python standard library. Zero dependencies. Same input always produces same output. Every fired rule is named and described.

3.2 The Five Detection Layers

Layer	What it catches & why
1 · Antonym-pair scanner (+40 pts)	Direct semantic opposites: stateless/stateful, never writes/writes to disk, safe indefinitely/should not. Highest weight — these are the clearest, most unambiguous contradictions.

2 · Absolute-negation flip (+35 pts)	A categorical claim (never, always, no X) followed by a contradicting claim on the same subject. Catches the transformer 'stateless' case.
3 · Quantity contradiction (+30 pts)	Zero/none vs non-zero quantity about the same subject. Also all/every vs sometimes/partial. Catches policy violations like 'never shared... shares with partners.'
4 · Scope-violation detector (+25 pts)	Universal scope (in all cases, without exception) followed by a scoped exception (sometimes, under certain conditions).
5 · Shared-subject negation (+20 pts)	Two sentences sharing 2+ key nouns where one asserts and the other denies. Lowest weight, fallback only. Does not fire if already covered by layers 1-4.

3.3 Scoring & Confidence Tiers

Score range	Meaning & recommended action
≥ 60 → HIGH	Strong contradiction. Multiple rules fired. Flag for correction before the answer reaches a user.
35–59 → MEDIUM	Possible contradiction. Route to human review, not auto-reject.
< 35 → LOW	Weak signal. May be nuance. Manual review only, do not auto-reject.
0 → CLEAN	No contradiction found. Note: does not mean factually correct — only logically consistent.

3.4 False-Positive Prevention — Most Important Constraint

The primary failure mode of any contradiction detector is over-flagging. I identified four false-positive scenarios and designed specifically to prevent each:

4. **Qualified statements:** 'X is generally true... but not always.' Fix: antonym pairs are specific ('safe for long-term', not just 'safe') so they only fire on categorical claims, not hedged ones.
5. **Temporal statements:** 'In 2020, policy was X. Today it is Y.' Fix: shared-keyword overlap check requires subject overlap — temporal statements often lack this.
6. **Domain idioms:** Technical terminology that looks contradictory out of context. Fix: pairs sourced from real technical domains, reviewed for specificity.

7. **Figurative language:** 'Both a blessing and a curse.' Fix: figurative phrases do not share concrete subject nouns with specific claims, so the overlap filter removes them.

Proof it works: Case 5 (ML data scaling nuance) scores 0 every single run. This is the key validation of the false-positive prevention.

4. Code Walkthrough — Every File Explained

4.1 detector.py — The Core Engine

The heart of the project. Five detection layers, scoring logic, and the prompt guide. No imports beyond Python standard library re module. Runs anywhere Python 3.10+ is installed.

ANTONYM_PAIRS — The Knowledge Base

The most important data structure. 40+ tuples of (phrase_a, phrase_b). The engine checks every sentence pair: sentence i has phrase_a, sentence j has phrase_b → +40 flag. Pairs are deliberately specific: 'safe for long-term' not just 'safe'. This specificity is the primary false-positive prevention.

_split_sentences() — Text Preprocessing

Splits on . ! ? \n boundaries. Filters fragments under 10 characters. Deliberately simple — no NLP libraries needed. The tradeoff is that mid-sentence punctuation (decimals in numbers) can occasionally cause wrong splits, but this is rare in the type of AI answers we analyze.

_shared_keywords() — The Subject Overlap Filter

Before most rules fire, we check that two sentences share at least one keyword longer than 4 characters. This is the primary nuance filter: sentences about completely different subjects cannot contradict each other. This single check prevents the majority of false positives.

detect_contradiction() — The Public API

Single public function. Takes a text string, runs all five layers, deduplicates findings via `_dedup()`, sums scores (capped at 100), determines confidence, selects best quote pair, returns structured dict. The deduplication prevents the same sentence pair from being counted twice across different rule layers.

4.2 main.py — CLI Interface

Flag	What it does
------	--------------

--check "text"	Runs detector on a single text. Prints JSON + human-readable summary with confidence, score, and conflicting quotes.
--test	Runs all 6 test cases. Prints pass/fail per case + final precision/recall/F1.
--guide	Prints the full prompt guide from detector.py.

4.3 test_cases.py — Six Hand-Crafted Test Cases

Case	Purpose
#1 Redis persistence (easy)	Baseline — obvious technical contradiction. 'Never writes to disk' vs 'writes to disk with AOF logs.'
#2 Python GIL (easy, clean)	Clean nuanced explanation. Must NOT be flagged. Tests basic false-positive protection.
#3 Transformer statefulness (hard)	The critical tricky case. 'Stateless, no memory' vs 'memory of earlier tokens.' Both halves authoritative. This is the case the brief specifically targets.
#4 Ibuprofen safety (medium)	Medical domain. 'Safe indefinitely' vs 'serious risks, do not exceed 10 days.' High real-world harm potential.
#5 ML data scaling (medium, clean)	False-positive trap. Diminishing returns is nuance, not contradiction. Must score 0.
#6 Privacy policy (medium)	'Never shared, no exceptions' vs 'shares with advertising partners.' Common in corporate AI outputs.

4.4 run_tests.py — Test Runner

Runs all six cases, calculates real metrics, saves test_results.json.

- **Precision:** of all flags, how many were real contradictions? Low precision = users stop trusting the tool.
- **Recall:** of all real contradictions, how many were caught? Low recall = dangerous for medical/legal contexts.
- **F1:** harmonic mean. Overall balance measure.

Actual test results

Precision: 1.00 · Recall: 1.00 · F1: 1.00 TP=4 FP=0 FN=0 TN=2 · 6/6 passed

4.5 contradiction_detector.html — Visual Demo

Same rule engine ported to JavaScript. Opens in any browser — no server, no API. Four tabs:

- **Detector** — textarea, example chips, animated score ring (green/amber/red), fired rules listed, conflicting sentences side by side
- **Test results** — runs all 6 cases live, shows precision/recall/F1, each case expandable
- **How it works** — explains all 5 layers and scoring visually
- **Prompt guide** — what is and is not a contradiction, tips, what the tool does not catch

4.6 Dockerfile

Minimal `python:3.11-slim` container. No pip install needed (zero dependencies). Defaults to prompt guide. Run `docker run contradiction-detector python main.py --test` for the test suite.

5. How It Solves the Problem

5.1 End-to-End Flow

8. Text received via CLI (`--check`) or programmatically via `detect_contradiction()`
9. `_split_sentences()` splits into individual sentences, filters fragments under 10 characters
10. Each sentence normalized — lowercased, quotes stripped — creating the pattern-matching surface
11. All five rule layers run in sequence, each outputting a list of findings with check type, sentence indices, and description
12. `_dedup()` removes duplicate findings (same pair caught by multiple rules)
13. Scores from all fired rules summed, capped at 100
14. `flag = True` if score ≥ 25 . Confidence: high ≥ 60 , medium 35–59, low < 35
15. Best quote pair extracted verbatim from the highest-scoring finding
16. Structured dict returned: `flag`, `score`, `confidence`, `checks list`, `quote_a`, `quote_b`, `reason`

5.2 Actual Test Results

Metric	Result
Cases passed	6 / 6

True Positives (TP)	4 — real contradictions correctly flagged
False Positives (FP)	0 — clean texts not incorrectly flagged
False Negatives (FN)	0 — no real contradiction missed
True Negatives (TN)	2 — clean texts correctly cleared
Precision	1.00 (100%)
Recall	1.00 (100%)
F1 Score	1.00

The key validation: Case 3 (tricky transformer text) scores 75/100 high confidence. Case 5 (nuance trap) scores 0. The engine catches the hard case and does not over-flag the nuanced one.

5.3 Reliability Analysis — Honest Limits

Included in the README because honest evaluation of limits is as important as demonstrating the tool works.

- Six test cases is a proof of concept, not a reliability measurement. A proper benchmark needs 200–500 labeled examples across domains, annotated by multiple human raters with inter-annotator agreement (Cohen's κ) measured.
- For a trust/safety tool, recall matters more than precision. Missed contradictions in medical or legal text can cause real harm. False alarms are annoying but not dangerous.
- Main false-positive risks: qualified statements, temporal statements, domain idioms, figurative language. The README explains how the engine handles each.
- Next steps: domain-specific antonym pair modules, 200-case benchmark, second-pass verification for low-confidence flags, ensemble approach (two independent rule passes, flag only if both agree).

6. Summary

What was built

A 5-layer rule-based self-contradiction detector. Pure Python, zero dependencies, no API key, runs offline and in the browser. Returns structured JSON with a 0-100 severity score, confidence tier, fired rules with descriptions, and verbatim conflicting sentences.

What makes it different from a simple keyword checker

The combination of five independent rule layers, the shared-keyword subject overlap filter (which prevents nuanced text from being flagged), and the specific antonym phrases (which avoid broad word matching false positives) is what makes it accurate. Case 5 scoring 0 on the nuance trap is the key evidence.

The tricky case — the heart of the task

Case 3 (transformer statefulness) is the core validation. It is exactly the 'confident, plausible, wrong' failure type the brief specified. Both halves sound authoritative, neither hedges, and the engine catches it at 75/100 high confidence via two independent rules.

Key learning

The most important thing was not about code. A detector with 95% accuracy used incorrectly is more dangerous than a 90%-accurate one used correctly. That is why the prompt guide, confidence tiers, and reliability analysis are as important as the detection engine itself.